

Real-Time Neural Signal Decoding System

Yibo Wang (yw4626) & Howard Wang (hw3128)

I. Problem Definition and Project Overview

Brain-computer interfaces (BCIs) aim to translate neural activity into meaningful control signals for external devices. In this project, our goal is to build a real-time neural decoding system that predicts five-finger movement trajectories from ECoG signals. ECoG, or electrocorticography, records neural activity directly from the cortical surface and provides higher spatial and temporal resolution than non-invasive EEG. This makes it more suitable for decoding fine motor movements such as individual finger motion.

The decoding problem in this project is a continuous regression task. The input is a recent multi-channel ECoG signal window, and the output is a five-dimensional vector representing the estimated positions or movements of the thumb, index, middle, ring, and little finger.

The dataset follows the BCICIV-style structure. The main variables include `train_data`, which stores the multi-channel ECoG signal; `train_dg`, which stores the five-finger ground-truth movement trajectories; and `test_data`, which contains unlabeled test ECoG signals. The signal is sampled at 1000 Hz, so each second contains 1000 ECoG samples. To decode finger movement in real time, the system uses a 300ms sliding window and produces one prediction every 50ms.

The overall system has two major stages:

- **Offline training:** In this stage, labeled ECoG data is loaded from .mat files, split into temporal training and validation segments, converted into sliding-window features, and used to train different machine learning models. The trained models are saved as *joblib* artifacts.
- **Online inference:** In this stage, ECoG samples are published into Pub/Sub, processed by an Apache Beam pipeline running on Google Cloud Dataflow, decoded using the trained model, and written to an output Pub/Sub topic.

The key design goal is to bridge offline neural decoding and real-time streaming inference. Offline training can directly access the full recording and build windows from complete arrays. However, in a real streaming setting, data arrives sample by sample. Therefore, the online system must maintain a stateful sliding window for each session and trigger model inference only when enough recent samples are available. In addition, in real-world application scenarios, it is necessary to jointly process the data of multiple subjects. Therefore, it needs to be deployed on the cloud.

II. Technical Design and Decoding Pipeline

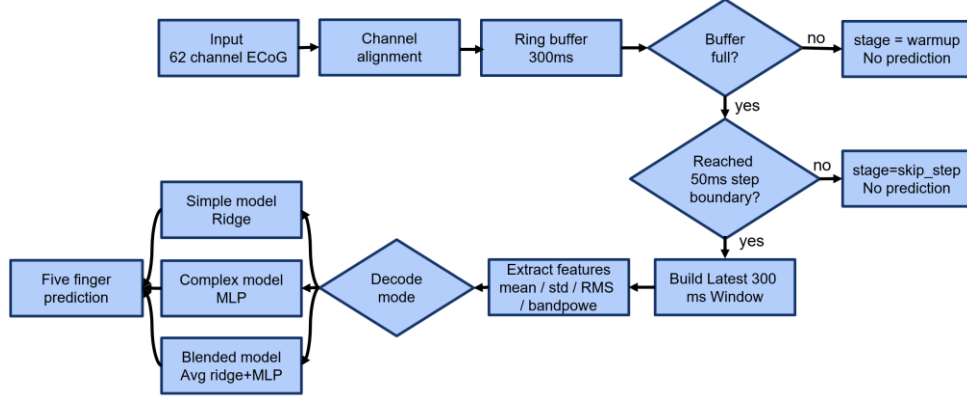


Fig 1. Decoding Pipeline

As for the feature engineering, the feature extraction pipeline converts raw ECoG signals into compact window-level features. Each input window contains the most recent 300ms of neural activity. For each channel, the system extracts time-domain features such as mean, standard deviation, and root mean square (RMS). RMS is useful because ECoG signals fluctuate around positive and negative values; unlike the mean, RMS captures the overall amplitude or energy of the signal without cancellation. The pipeline also extracts frequency-domain band power features, including alpha, beta, low-gamma, and high-gamma bands. High-gamma activity is especially important for motor decoding because it is often associated with fine motor movement.

Several models were explored, including Ridge Regression, Random Forest, MLP, and Hoeffding Tree regression. Ridge Regression is lightweight and fast, making it suitable for streaming inference. MLP provides stronger nonlinear modeling capability and achieved better offline decoding accuracy in our experiments. Random Forest is also expressive but can be less efficient in real-time inference. Hoeffding Tree is a streaming-friendly model because it supports incremental learning, although its decoding accuracy was not the best in our current setup. Eventually, Ridge Regression was chosen for speed, stability, and streaming efficiency, while MLP was chosen for stronger nonlinear modeling and better decoding accuracy, and also combine the averaged output in both model as the blended model, aims to overcome the bias. Together, they allow us to study the trade-off between real-time efficiency and prediction performance.

The real-time decoding core is implemented through a BCIDecodePipeline, can be viewed in Fig 1. Each incoming ECoG sample is first aligned to the expected channel count.

Algorithm 1 Ring-buffer sliding-window decoding (`BCIDecodePipeline.push`)

Param : $W \in \mathbb{Z}^+$, window length (samples per window).

Param : $S \in \mathbb{Z}^+$, decode stride (prediction every S -th arrival).

Param : $C \in \mathbb{Z}^+$, channel count after alignment.

Param : $\text{Align}(\mathbf{x}) \mapsto \mathbf{r} \in \mathbb{R}^C$.

Param : $\text{Feat}(\mathbf{W})$, feature extractor on ordered window.

Param : $\text{Predict}(\mathbf{f})$, predictor on feature vector $\mathbf{f}$.

```
1: State: buffer  $\mathbf{B} \in \mathbb{R}^{W \times C}$ ; fill  $f \in \{0, \dots, W\}$ ; cursor  $s \in \{0, \dots, W-1\}$ ;  
   arrival counter  $t$ .  
2: Init:  $\mathbf{B} \leftarrow \mathbf{0}$ ,  $f \leftarrow 0$ ,  $s \leftarrow 0$ ,  $t \leftarrow 0$   
3: while not end of stream do  
4:   read incoming sample  $\mathbf{x}_t$   
5:    $\mathbf{r}_t \leftarrow \text{Align}(\mathbf{x}_t)$   
6:    $t \leftarrow t + 1$   
7:   if  $f < W$  then  
8:      $\mathbf{B}_{f,:} \leftarrow \mathbf{r}_t^\top$  ▷ ▷ warmup: sequential rows until length  $W$   
9:      $f \leftarrow f + 1$   
10:  else  
11:     $\mathbf{B}_{s,:} \leftarrow \mathbf{r}_t^\top$  ▷ ▷ overwrite oldest logical row (slide window)  
12:     $s \leftarrow (s + 1) \bmod W$   
13:    if  $t \bmod S = 0$  then  
14:       $\mathbf{W}_{\text{ord}} \leftarrow \text{ORDERED}(\mathbf{B}, s)$  ▷ ▷ chronological matrix, size  $W \times C$   
15:       $\mathbf{f} \leftarrow \text{Feat}(\mathbf{W}_{\text{ord}})$   
16:       $\hat{\mathbf{y}} \leftarrow \text{Predict}(\mathbf{f})$  ▷ ▷ emit decoded output  
17:    end if  
18:  end if  
19: end while  
  
20: procedure  $\text{ORDERED}(\mathbf{B}, s)$   
21:   if  $s = 0$  then  
22:     return  $\mathbf{B}$   
23:   else  
24:     return  $[\mathbf{B}_{s:W-1,:}^\top \mid \mathbf{B}_{0:s-1,:}^\top]^\top$  ▷ ▷ tail || head in time order  
25:   end if  
26: end procedure
```

Then, there is a specific streaming algorithm called sliding window. The sample is inserted into a ring buffer that stores the latest 300ms of data. A ring buffer is used because it has fixed memory size and updates efficiently by overwriting the oldest sample instead of shifting the entire buffer. This is important for high-frequency streaming input. The decoder first enters a warmup stage until the ring buffer contains a full 300ms ECoG window. After that, every incoming sample updates the buffer, but prediction is only triggered at each 50ms step boundary.

The system supports three modes: simple mode uses Ridge for fast inference, complex mode uses MLP for stronger nonlinear decoding, and blended mode combines both predictions with a weighted average. The main technical challenge is balancing decoding accuracy and streaming efficiency, since MLP can improve prediction quality but Ridge is faster, lighter, and easier to scale in Dataflow.

III. Streaming and Distributed Processing with Dataflow

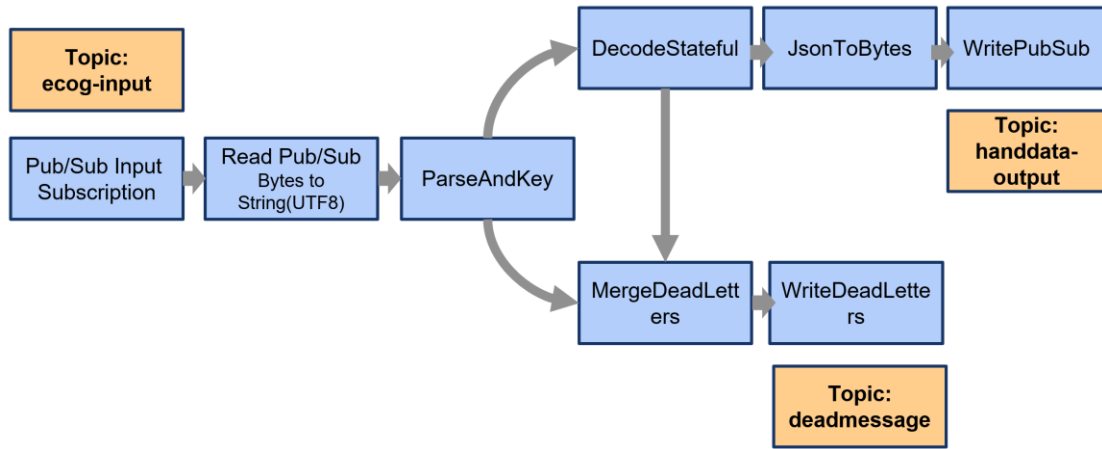


Fig 2. Dataflow Architecture

We used Google Pub/Sub as the streaming message layer, Apache Beam as the programming model, and Google Cloud Dataflow as the managed distributed execution engine. The dataflow architecture can be viewed in Fig 2.

The streaming input message contains a `session_id`, a `channels` array, and an optional `sample_seq` field. The `channels` array represents one time point of multi-channel ECoG data. Pub/Sub transmits message payloads as bytes, so the Beam pipeline first reads bytes from Pub/Sub and decodes them into UTF-8 strings. The JSON string is then parsed into a Python dictionary. The pipeline extracts `session_id` and converts the element into a keyed value pair: (`session_id`, `payload`). This keyed structure is essential because the decoder must maintain a separate sliding window for each session. Different sessions cannot share the same buffer, otherwise samples from different neural streams would be mixed together and the model input would become invalid.

The core streaming operator is `DecodeStateful`. Which is the decode pipeline we mentioned before. If the data is input in parallel, the worker will automatically create the corresponding number of decoding pipelines.

Distributed processing is exploited through multiple `session_id` keys. A single session must be processed sequentially because its sliding window depends on temporal order. However, different sessions are independent and can be processed in parallel by Dataflow workers. This is why increasing parallel sessions can increase total throughput. In our scalability tests, throughput generally increased as the number of parallel sessions increased, although the system eventually reached a resource contention point where latency began to rise.

The pipeline also includes a deadletter path. Invalid JSON, missing channels, non-numeric channel values, invalid `sample_seq`, or out-of-order samples are sent to a deadletter topic instead of crashing the streaming job. This improves robustness and makes debugging easier.

IV. Deployment Challenges and Dataflow Debugging

The first Dataflow deployment issue was the public IP restriction. Since workers were launched without external IPs, they initially could not access required Google services such as Cloud Storage and Pub/Sub. We resolved this by enabling Private Google Access on the subnet, allowing private workers to reach Google APIs securely without public IPs.

The second major issue was that Dataflow workers could not connect correctly at the beginning. In practice, this appeared as worker startup or connection failures. We changed the worker machine type. The original worker configuration used an older n1 machine type, and we later switched to a newer n2 worker type. This improved worker startup and connectivity behavior; it even improves the throughput performance.

The main challenge in model selection was balancing decoding accuracy with streaming efficiency. Ridge Regression is fast, stable, and easy to deploy in Dataflow, but its linear structure limits prediction accuracy. MLP can capture more complex nonlinear relationships between ECoG features and finger movements, giving better decoding performance, but it introduces higher inference cost and more tuning complexity. Random Forest and Hoeffding Tree were also considered: Random Forest is strong for offline nonlinear modeling but heavier for real-time deployment, while Hoeffding Tree is streaming-friendly but less accurate in our experiments. Therefore, we used Ridge as the lightweight baseline, MLP as the accuracy-oriented model, and blended Ridge+MLP as a compromise between prediction quality and real-time processing efficiency.

V. Limitations and Future Work

One limitation is potential data leakage because some tests replay data derived from the training recording, so performance may be overestimated. Also, the current parallel-session test reuses one subject's .joblib model, so it simulates multiple sessions rather than true multi-subject input. In real use, each subject may need a separately trained or calibrated model, otherwise cross-subject prediction accuracy would likely drop.

Also, the input is simulated real-time data rather than live neural acquisition. The current streaming input is generated by replaying .mat files into Pub/Sub. This is useful for testing the streaming architecture, but a real BCI system would need integration with an actual ECoG acquisition device.

Future work can optimize inference using ONNX Runtime or other lightweight serving formats instead of Python sklearn models. This could reduce model prediction latency and make deployment more efficient. Beam state overhead could also be reduced through better serialization, compression, or carefully designed micro-batching that preserves temporal order.

In addition, in the future, the training process might also be handled in the cloud, using ExStream or REMIND as deep learning methods. This would save the offline training step, but it would significantly increase the deployment complexity and cloud costs.